

Tyler Vander Mooren

Team Solo 2

CYB 310: Malware Analysis

Professor Troy Adams

April 27, 2026

Team Assignment 2: Capstone – End-to-End Malware Analysis

Malware Identification and Classification

The analyzed sample is a malicious Microsoft Excel macro-enabled file that was retrieved from MalwareBazaar. The identifying details are listed below.

- **Malware sample name:** tn4598151.xlsm
- **Source Repository:** MalwareBazaar
- **SHA-256 hash:**

500856ee3fc13326cad564894a0423e0583154ef10531de4ab6e6d5df90d4e31

Further inspection of the Excel worksheet shows it contains an embedded VBA project file named vbaProject.bin, which confirms that the document contains macro code. In addition, it contains a Workbook_Open auto execution routine, suspicious VBA keywords, formula-based string construction and attempted staging of a DLL payload. These findings confirm the sample is malicious and its behavior is most consistent with a first stage downloader/dropper, where the Office document is used to initiate code execution and prepare the secondary payload.

Static Analysis Findings

Static analysis was conducted on the Office document container and extracted the vbaProject.bin using 7-zip. The presence of vbaProject.bin confirms that the workbook contains embedded VBA macro code. When oleid was used, it identified the file as an OLE/Compound File container and showed that it was not encrypted. More importantly it flagged the presence of the VBA macro as suspicious with a high-risk rating and recommended deeper analysis. The use

of olevba provided the strongest static evidence. It was able to identify a macro stream in ThisWorkbook.cls and showed the Workbook_Open() routine. This is a significant find because Workbook_Open runs automatically when Excel is opened, so this is a commonly used execution trigger in malicious Office documents. The macro also performed several suspicious actions, including checking the environment variable NUMBER_OF_PROCESSORS, which may indicate a basic environment or sandbox check. Also, the macro used Environ("LocalAppData") to locate the user's local application data folder, create a think-cell directory and then would reference test.dll, likely indicating that this is the staged or downloaded payload.

Further analysis showed that inside the workbook it used string and formula obfuscation. The macro wrote command fragments into worksheet cells, which include pieces such as Down, oFileA, ll32, llex, run, tp and //. These fragments appear designed to reconstruct larger commands such as download and execution without storing the complete command plainly in one location. The macro then created a new sheet, defined a named cell called asd, wrote formulas into the worksheet cells and executed the named cell with Run "asd". Also, when the worksheet is opened in Excel the user is instructed to press "Enable Editing" and "Enable Content". This is a clear social engineering lure intended to convince the user to allow the macro execution. The worksheet is visibly blocked by this huge message and upon inspection of the individual cells behind the message, cells in A10-A25 contain CONCATENATE formulas pointing to scattered cell locations. This further supports the conclusion that the malicious file used worksheet-based obfuscation to hide command strings.

Table 1

Olevba summary of suspicious indicators.

Indicator	Meaning
-----------	---------

Workbook_Open	Macro runs when the workbook opens
Environ	Reads system environment variables
open	May open a file
run	May execute a command or program
Call	May call DLL or Excel 4 macro functionality
Mkdir	Creates a directory
Hex strings	Possible obfuscation
Base64 strings	Possible obfuscation
test.dll	Executable/DLL filename IOC

Dynamic and Behavioral Analysis

When dynamic analysis was conducted the document would successfully initiate stage one of the macro behavior but the full payload chain was not observed during the dynamic runs. Observation was first performed on a Windows 10 virtual machine using ProcMon and Wireshark and then later using ANY.RUN sandbox output as supporting evidence. The most important runtime finding was that Excel created the directory think-cell. This matches the VBA macro code inspection, so confirming the macro at least executed to this part of its staging logic during dynamic analysis. However, confirmation of the creation of test.dll was not observed nor did it show a clear process like rundll32.exe, cmd.exe or powershell.exe spawning from the Excel sheet. ANY.RUN was able to provide additional context in that observation was observed up to the creation of dwwin.exe but then runs into a Windows error rather than completing the second stage payload run. Also, the report lists malware activity behavior directly to the Excel macro activity, so although the secondary payload was not confirmed, the static evidence and observed staging behavior are enough to conclude that the file is malicious.

Indicators of Compromise

During local VM testing, Wireshark did not capture a clear outbound malware callback from Excel or confirm a second stage payload connection. There are many reasons this could have failed, payload URL not available, macro execution failing after staging, the VM environment was not suitable or the malware suppressed any visible errors. A rather interesting datapoint is also retrieved by the malware, which is the number of processors, this could indicate it is trying to see if the environment can support it or an attempt to detect VM environments. This is further supported by the report from ANY.RUN which during execution the Excel worksheet would error, indicating the possibility of the execution being too heavy. Furthermore, ANY.RUN did record some network activity but these appeared to be mostly normal Windows or Microsoft traffic. So, network evidence was not very conclusive in comparison to static evidence. The main IOCs are host and file based rather than network based.

Table 2

Observed indicators of compromise

Indicator Type	Value	Confidence	Source
SHA-256	500856ee3fc13326cad564894a0423e0583154ef10531de4ab6e6d5df90d4e31	High	Sample
File type	Macro-enabled Excel worksheet	High	Static inspection
Embedded macro-object	vbaProject.bin	High	7-zip / oletools

Auto	Workbook_Open	High	olevba
execution			
macro			
Local	\\labrat\AppData\Local\think-cell	High	Static /
observed			ProcMon
path and			
staging			
directory			
Reference	%LocalAppData%\think-cell\test.dll	Medium	olevba
payload and			
path			
Process	dwwin.exe	Medium	ANY.RUN
Social	Fake Office 365 “Enable Editing / Enable Content” Message	High	Manual
engineering			workbook
lure			inspection
Obfuscation	CONCATENATE formulas and scattered cell strings	High	Manual
method			workbook
			inspection

Defensive Recommendations

Based on the information gathered from analysis defensive response should focus on preventing malicious Office macro execution and detecting suspicious Office behavior on endpoints. Organizations should block or quarantine the known SHA-256 hash of this sample. Further recommendations should flag or block macro-enabled Office attachments from untrusted

senders and Office macro execution should be disabled or restricted for documents downloaded from the Internet. Endpoint detection should monitor for Office applications that are creating folders, writing executable files or launching child processes from user-writable paths. In the case of this malware the specific target should be %LocalAppData%\think-cell and test.dll. In addition, security teams should also create detections specifically for Office process chains, which include:

Table 3

Suspicious behaviors to monitor

Suspicious behavior	Defensive value
EXCEL.exe creating directories in %LocalAppData%	Detects staging behavior
EXCEL.exe writing .dll files	Detects payload placement
EXCEL.EXE spawning rundll32.exe	Detects DLL execution attempts
Excel.exe spawning cmd.exe or powershell.exe	Detects command execution
Office documents using auto execution macros	Detects malicious document behavior
Office documents containing obfuscated formulas	Detects macro staging and evasion

User awareness training should also address fake document lures. The visible Office 365 themed message in this sample was an attempt to convince the user to enable editing and content, which allowed the macro to run.

Limitations and Uncertainty

A key limitation in this analysis was the incomplete second stage payload execution. Static analysis was able to strongly confirm that the workbook contains malicious macro logic and references a staged DLL payload. Dynamic analysis confirmed the creation of the think-cell staging directory but was unable to confirm the successful creation or execution of test.dll. The macro also utilized On Error Resume Next, which suppresses visible runtime errors. Because of this, the macro may have failed during the download or execution but it doesn't show any errors because of the suppression. So, the error may have been caused by unavailable external infrastructure, environmental requirements, sandbox/VM detection or the payload URL no longer being active. Despite not observing second stage execution confidence is high that the document is malicious and that it performs macro-based staging behavior. Confidence is low however about the final payload and its capabilities since it was never fully recovered and analyzed.

The analyzed sample is a malicious Excel macro-enabled worksheet that uses social engineering, auto executing VBA, formula-based string obfuscation and local staging behavior to attempt delivery or execution of a secondary DLL payload. Static analysis was able to confirm this, while dynamic analysis confirmed the creation of the expected think-cell staging directory proving that the macro executed at least part of its payload logic. Overall, the conclusion is that the sample is a malicious macro downloader/dropper with confirmed first stage behavior and unconfirmed final payload execution.